



Introduzione

Corso di Laboratorio di Gestione progetto e organizzazione d'impresa

Prof. Dei Rossi, Leonardo Essam

Perché studiare GPOI?

Perché studiare GPOI? (1)

1. **Il software non è mai un'opera solitaria**

Nel mondo del lavoro il codice nasce quasi sempre in team, non *"ogniuno il suo"*.

2. **Progetti complessi $\iff$ più persone**

Funzioni diverse, competenze diverse $\Rightarrow$ servono coordinamento e regole comuni.

3. **Collaborazione e responsabilità**

Imparare a lavorare con altri sviluppatori, condividere risorse, gestire tempi e priorità.

4. **Strumenti e metodi**

Version control, task board, planning: senza un metodo il caos è dietro l'angolo.

5. **Competenze trasversali**

Saper comunicare, pianificare e organizzare è ciò che distingue un buon tecnico da un professionista.

Perché studiare GPOI? (2)

A scuola:

- Ogni studente sviluppa il proprio codice;
- Progetti di piccola scala;
- Scadenze flessibili e valutazione individuale;
- Focus sulle competenze tecniche;
- Correzioni del prof, poche revisioni tra pari;
- Documentazione minima.

Al lavoro:

- Più persone lavorano sullo stesso progetto;
- Progetti complessi e duraturi;
- Deadline rigide, obiettivi condivisi;
- Focus su competenze tecniche + organizzative + comunicative;
- Code review, test e qualità condivisa;
- Documentazione dettagliata per il team e per il cliente.

Nascita dello Sviluppo Collaborativo

Storicamente, lo sviluppo del software ha richiesto pochissima collaborazione. Con l'avvento di **Internet**, sono arrivate nuove opportunità, ma anche una **serie di problemi completamente nuova**.

Costretti dal bisogno e dall'economia a sviluppare la propria infrastruttura software, i pionieri di internet come **Amazon**, **Facebook**, **Google**, **LinkedIn** e **Twitter** hanno evoluto non solo software diversi, ma anche atteggiamenti distinti verso il suo valore.

Cambiamento di atteggiamento verso software (1)

1. **1960:** IBM: "Software serve per vendere hardware";
2. **1981:** Microsoft: "Software serve per fare soldi";
3. **1994:** Amazon: "Software viene usato per servizi che fanno soldi e meritano di essere protetti";
4. **2004:** Facebook: "Software è usato per servizi che fanno soldi e non vale la pena proteggere".

Cambiamento di atteggiamento verso software (2)

Come cambierà ancora?

"Se (a) siamo tutti a sviluppare software simili, e (b) non è competitivo differenziare, lo dobbiamo sviluppare insieme."

(O'Grady 2015)

Sistemi di versionamento

Version Control Systems (VCS) (1)

Un **sistema di versioning** è un software che tiene traccia di tutte le modifiche¹ fatte ai file di un progetto e permette di tornare indietro a versioni precedenti se qualcosa non funziona².

Aiuta più persone a lavorare sugli stessi file senza perdere il lavoro degli altri.

Senza versioning il rischio è di sovrascrivere i file o di perdere il lavoro (in tutti i sensi!).

Con il versioning ogni modifica ha: **un autore, una data e uno storico di modifiche.**

¹Vita, morte e miracoli

²Macchina del tempo

Version Control Systems (VCS) (2)

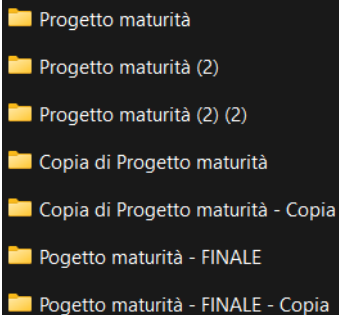
Alcune parole chiave:

- **Commit:** salvataggio di uno "scatto" di uno o più file;
- **Branch:** "copia" separata per lavorare senza interferire con il codice principale;
- **Merge:** unione tra le varie "copie" e il codice principale.

Patti chiari, amicizia lunga: non voglio sentire *traduzioni* fantasiose (i.e. *committare*, *branchare* e *mergiare*); siete dei tecnici e da voi ci si aspetta un linguaggio tecnico!

Version Control Systems (VCS) (3)

Perché usare un sistema di versionamento?



- Progetto maturità
- Progetto maturità (2)
- Progetto maturità (2) (2)
- Copia di Progetto maturità
- Copia di Progetto maturità - Copia
- Pogetto maturità - FINALE
- Pogetto maturità - FINALE - Copia

Per evitare situazioni come queste (episodio realmente accaduto...)!



Controllo di versione con Git

Git (1)

Git è un software per il controllo di versione distribuito utilizzabile da interfaccia a riga di comando (command prompt).

Git³ nasce per essere uno strumento semplice per facilitare lo sviluppo del kernel Linux. La sua progettazione si ispirò ad analoghi strumenti (al tempo, software proprietari) come BitKeeper e Monotone.

È stato originariamente creato da Linus Torvalds (il padre di Linux, ndr) e pubblicato per la prima volta l'8 aprile 2005.

Git è disponibile per GNU/Linux, macOS, Windows e Solaris.

³Nello slang britannico, *Git* significa *idiot*

Git (2)

Git è un sistema distribuito.

Questo significa che ogni sviluppatore ha una **copia completa** del repository (codice + cronologia).

Il vantaggio è che non esiste un *single point-of-failure*!

Esistono altri sistemi VCS che sono invece centralizzati: ovvero che il codice è ospitato su un unico server, ma non li tratteremo in questo corso.

**Quali sono i possibili vantaggi di un
VCS distribuito?**

Vantaggi di Git (o di un generico VCS distribuito)

- Ogni sviluppatore ha l'intero progetto con tutta la storia sul proprio PC;
- Si può lavorare anche senza connessione internet e sincronizzarsi dopo;
- Non esiste un punto unico di fallimento: se il server remoto cade, il progetto non si perde;
- Operazioni come *commit*, *branch* e *merge* sono più veloci perché avvengono in locale
- È facile creare rami (*branches*) paralleli per nuove funzioni e poi unirli;
- Si può ripristinare una copia completa del progetto da qualsiasi sviluppatore, in caso di emergenza.

